Music Mood DB: ER to Relational Schema Mapping

DBMS Final Project, Phase 1. This document shows, step by step, how each entity, attribute and relationship in the v2 ER diagram was converted into the 19 tables in `music_mood_db.sql`.

1. The mapping rules we applied

#	ER construct	Relational rule
R1	Strong entity	One table. Simple attributes become columns. The key attribute becomes the PRIMARY KEY.
R2	Weak entity	One table. PK = owner's PK + the partial key (discriminator). The owner's PK is also a FK (ON DELETE CASCADE).
R3	Composite attribute	Flatten into its simple parts. The composite itself is not stored.
R4	Multivalued attribute	Separate table. PK = (owner PK, the value). FK back to the owner.
R5	Derived attribute	Not stored. Computed with a function or view.
R6	1:N relationship	Put the PK of the 1-side into the N-side table as a FK. Relationship attributes move to the N-side.
R7	M:N relationship	New table. PK = combination of both FKs. Relationship attributes become columns.
R8	Recursive relationship	Same rules as R6/R7, but both FKs point to the same table, with role names.
R9	Ternary relationship	New table with three FKs. We added a surrogate key `rec_id`.
R10	ISA (specialization)	One table per subtype. Its PK is also a FK to the supertype. Only subtype-specific attributes live there.
R11	Total participation	The FK column is NOT NULL.

2. Strong entities (R1)

Each strong entity box in the diagram becomes one table. The underlined attribute is the PK.

Entity	Table	Primary key	Other notes
ARTIST	`artist`	`artist_id`	`name`, `country`, `debut_year`
ALBUM	`album`	`album_id`	`artist_id` FK added by `releases`
TRACK	`track`	`track_id`	`valence` / `energy` have CHECK 0..1. UNIQUE(`album_id`, `track_no`)
GENRE	`genre`	`genre_id`	`name` is a UK
MOOD	`mood`	`mood_id`	`name` is a UK. `parent_mood_id` added by `parent_of`
APP_USER	`app_user`	`user_id`	`username` and `email` are UKs (candidate keys)
USER_MOOD_LOG	`user_mood_log`	`log_id`	`user_id`, `mood_id` FKs added by `gives` and `describes`
LISTENING_HISTORY	`listening_history`	`listen_id`	`user_id`, `track_id` FKs added by `streams` and `listened_to`

Note: the entity `USER` was renamed `APP_USER` because USER is a MySQL keyword.

3. Weak entity (R2): PLAYLIST

A playlist number only makes sense inside one user, so PLAYLIST is a weak entity and `owns` is its identifying relationship.

```
CREATE TABLE playlist (
  user_id     INT,
  playlist_no INT,            -- partial key (discriminator)
  name        VARCHAR(100) NOT NULL,
  created_at  DATETIME NOT NULL,
  mood_id     INT NULL,       -- from themed_for (optional)
  PRIMARY KEY (user_id, playlist_no),
  FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE CASCADE
);
```

- PK = owner's key (`user_id`) + discriminator (`playlist_no`).
- Total participation of the weak entity in `owns` is why `user_id` can never be NULL.
- ON DELETE CASCADE: deleting a user deletes their playlists.

4. Attribute kinds

4.1 Composite attribute (R3): `name` in APP_USER

The diagram shows `name` split into `first_name` and `last_name`. Only the parts become columns.

```
name (first_name, last_name)  ->  first_name VARCHAR(40), last_name
VARCHAR(40)
```

4.2 Multivalued attribute (R4): `{alias}` in ARTIST

An artist can have many aliases, and a column cannot hold a list in 1NF, so it becomes its own table.

```
CREATE TABLE artist_alias (
  artist_id INT,
  alias     VARCHAR(100),
  PRIMARY KEY (artist_id, alias),
  FOREIGN KEY (artist_id) REFERENCES artist(artist_id) ON DELETE CASCADE
);
```

4.3 Derived attributes (R5)

None of these are columns. Each is computed on demand.

Derived attribute	Entity	How it is computed
`age()`	APP_USER	`fn_age(user_id)` using `TIMESTAMPDIFF` on `birth_date`
`play_count()`	TRACK	`fn_play_count(track_id)` : COUNT of `listening_history` rows
`completion_pct()`	LISTENING_HISTORY	Column in view `v_listening_detail` : listened sec / track `duration_sec` x 100
`track_count()`	PLAYLIST	`fn_track_count(user_id, playlist_no)`
`total_duration()`	PLAYLIST	`fn_total_duration(user_id, playlist_no)` : SUM of track durations

Why not store them? Stored copies can go out of date and would break 3NF (they depend on other data, not on the key alone).

5. One-to-many relationships (R6)

No new table. The FK goes into the **N-side** table. Where the N-side has total participation (double line), the FK is `NOT NULL`.

Relationship	1-side	N-side	FK added	NOT NULL?	Reason
`releases`	ARTIST	ALBUM	`album.artist_id`	Yes	ALBUM total
`contains`	ALBUM	TRACK	`track.album_id`	Yes	TRACK total. A single counts as a one-track album
`gives`	APP_USER	USER_MOOD_LOG	`user_mood_log.user_id`	Yes	LOG total
`describes`	MOOD	USER_MOOD_LOG	`user_mood_log.mood_id`	Yes	LOG total
`streams`	APP_USER	LISTENING_HISTORY	`listening_history.user_id`	Yes	LISTENING_HISTORY total
`listened_to`	TRACK	LISTENING_HISTORY	`listening_history.track_id`	Yes	LISTENING_HISTORY total
`owns`	APP_USER	PLAYLIST	`playlist.user_id`	Yes	Weak entity (part of PK)
`themed_for`	MOOD	PLAYLIST	`playlist.mood_id`	**No (NULL)**	Partial participation: a playlist may have no mood

The only nullable FK among these is `playlist.mood_id`, because that is the only 1:N link with partial participation on the N-side.

6. Many-to-many relationships (R7)

Each M:N relationship becomes its own table. The PK is the pair of FKs, which also prevents duplicate pairs. Relationship attributes (the small boxes joined by a dotted line) become ordinary columns.

Relationship	New table	PK	Relationship attributes
`performs` (ARTIST, TRACK)	`track_artist`	(`track_id`, `artist_id`)	`role`
`classified_as` (TRACK, GENRE)	`track_genre`	(`track_id`, `genre_id`)	none
`tagged_with` (TRACK, MOOD)	`track_mood`	(`track_id`, `mood_id`)	`confidence`
`rates` (APP_USER, TRACK)	`rating`	(`user_id`, `track_id`)	`rating`, `rated_at`
`includes` (PLAYLIST, TRACK)	`playlist_track`	(`user_id`, `playlist_no`, `track_id`)	`position`, `added_at`
`follows` (APP_USER, APP_USER)	`user_follow`	(`follower_id`, `followee_id`)	`followed_at`

Example, with the relationship attribute `confidence`:

```
CREATE TABLE track_mood (
  track_id   INT,
  mood_id    INT,
  confidence FLOAT NOT NULL DEFAULT 1,
  PRIMARY KEY (track_id, mood_id),
  FOREIGN KEY (track_id) REFERENCES track(track_id) ON DELETE CASCADE,
  FOREIGN KEY (mood_id)  REFERENCES mood(mood_id)   ON DELETE CASCADE
);
```

Composite foreign key in `includes`

Because PLAYLIST is a weak entity with a two-column key, `playlist_track` needs a **composite FK**:

```
FOREIGN KEY (user_id, playlist_no) REFERENCES playlist(user_id, playlist_no)
```

7. Recursive relationships (R8)

Both ends of the relationship are the same entity, so both FKs point to the same table. The role names written beside the lines in the diagram name the columns.

Relationship	Cardinality	Implementation
`parent_of` (MOOD, roles: parent / child)	1 : N	Self-referencing FK `mood.parent_mood_id -> mood.mood_id`, nullable because top-level moods have no parent
`follows` (APP_USER, roles: follower / followee)	M : N	Table `user_follow` with `follower_id` and `followee_id`, both FKs to `app_user`, plus CHECK `follower_id <> followee_id`

These structures are queried with recursive CTEs (mood hierarchy, follow graph).

8. Ternary relationship (R9): `recommends`

`recommends` connects APP_USER, TRACK and MOOD (M : N : P) and carries its own attributes (`score`, `generated_at`). It maps to one table with three FKs.

```
CREATE TABLE recommendation (
  rec_id       INT AUTO_INCREMENT PRIMARY KEY,   -- surrogate key
  user_id      INT NOT NULL,
  track_id     INT NOT NULL,
  mood_id      INT NOT NULL,
  score        FLOAT NOT NULL,
  generated_at DATETIME NOT NULL,
  FOREIGN KEY (user_id)  REFERENCES app_user(user_id),
  FOREIGN KEY (track_id) REFERENCES track(track_id),
  FOREIGN KEY (mood_id)  REFERENCES mood(mood_id)
);
```

Why a surrogate key `rec_id`? The natural key (`user_id`, `track_id`, `mood_id`, `generated_at`) is long, and the same combination can be recommended again at a later time. `rec_id` is shown as a key attribute on the diamond.

9. ISA specialization (R10)

APP_USER is specialized into FREE_USER and PREMIUM_USER (**disjoint, total**).

Table	PK	Subtype-only attributes
`free_user`	`user_id` (also FK to `app_user`)	`daily_skip_limit`
`premium_user`	`user_id` (also FK to `app_user`)	`plan_type`, `subscribed_on`, `valid_till`

- Shared attributes (`username`, `email`, …) stay in `app_user` only.
- **Disjoint** is enforced by triggers: inserting into one subtype fails if the user is already in the other.
- **Total** (every user must be in a subtype) cannot be enforced by a plain constraint in MySQL. It is handled by the application, a stored procedure or a trigger on `app_user`.
- A CHECK on `premium_user` enforces `valid_till >= subscribed_on`.

10. Participation and cardinality in the schema

Diagram feature	Where it shows up in SQL
Total participation (double line)	`NOT NULL` on the FK
Partial participation	Nullable FK (`mood.parent_mood_id`, `playlist.mood_id`) or a separate relationship table row that may not exist
1 : N	FK on the N-side
M : N	Composite PK in a relationship table
Weak entity	Composite PK containing the owner's key + ON DELETE CASCADE
Candidate keys (UK)	`UNIQUE` constraints
Value ranges	`CHECK` (rating 1-5, intensity 1-5, valence/energy/confidence/score 0..1)

11. Final result: 19 tables

How the diagram's 11 entities, 1 multivalued attribute, 6 M:N relationships and 1 ternary relationship add up:

#	Table	Comes from	Rule
1	`app_user`	Entity APP_USER	R1
2	`free_user`	ISA subtype	R10
3	`premium_user`	ISA subtype	R10
4	`artist`	Entity ARTIST	R1
5	`artist_alias`	Multivalued `{alias}`	R4
6	`album`	Entity ALBUM + `releases`	R1, R6
7	`track`	Entity TRACK + `contains`	R1, R6
8	`genre`	Entity GENRE	R1
9	`mood`	Entity MOOD + `parent_of`	R1, R8
10	`track_artist`	`performs`	R7
11	`track_genre`	`classified_as`	R7
12	`track_mood`	`tagged_with`	R7
13	`user_mood_log`	Entity + `gives` + `describes`	R1, R6
14	`listening_history`	Entity + `streams` + `listened_to`	R1, R6
15	`rating`	`rates`	R7
16	`user_follow`	`follows` (recursive)	R7, R8
17	`playlist`	Weak entity + `owns` + `themed_for`	R2, R6
18	`playlist_track`	`includes`	R7
19	`recommendation`	Ternary `recommends`	R9

Relationships that did **not** get their own table (`releases` , `contains` , `gives` , `describes` , `streams` , `listened_to` , `owns` , `themed_for` , `parent_of`) were folded into an existing table as a foreign key.

12. Next step: normalization

This mapping is the starting schema for the normalization report. The composite-key tables (`track_artist` , `track_mood` , `rating` , `playlist_track`) are good places to look for partial dependencies when checking 2NF, 3NF and BCNF.